

Threads syntax

```

1.  #include <iostream>
2.  #include <thread>
3.
4.  a. void task(){
5.      std::cout << "Hello from the thread\n";
6.  }
7.
8.  int main(){
9.      std::thread thread(task);
10.     thread.join();
11.     return 0;
12. }
```

①

```

1.  #include <iostream>
2.  #include <thread>
3.
4.  void add(int a, int b){
5.      std::cout << a + b << std::endl;
6.  }
7.
8.  int main(){
9.      std::thread thread(add, 10, 20);
10.
11.     thread.join();
12.
13.     return 0;
14. }
```

②

```

1.  #include <iostream>
2.  #include <thread>
3.
4.  void increment(int& number){
5.      number++;
6.  }
7.
8.  int main(){
9.      int number = 10;
10.     std::thread thread(increment, std::ref(number));
11.     thread.join();
12.     std::cout << number << std::endl;
13.     return 0;
14. }
```

③

```

1.  #include <iostream>
2.  #include <thread>
3.
4.  void task(int id){
5.      std::cout << "Thread " << id << std::endl;
6.  }
7.
8.  int main(){
9.      const int numThreads = 4;
10.     std::thread threads[numThreads];
11.     for (int i = 0; i < numThreads; ++i) {
12.         threads[i] = std::thread(task, i);
13.     }
14.
15.     for (int i = 0; i < numThreads; ++i) {
16.         threads[i].join();
17.     }
18.     return 0;
19. }
```

④

```

1.  #include <iostream>
2.  #include <thread>
3.  #include <mutex>
4.
5.  int counter = 0;
6.
7.  std::mutex mutexCounter;
8.
9.  void increment(){
10.     for (int i = 0; i < 100000; ++i) {
11.
12.         mutexCounter.lock();
13.
14.         counter++;
15.
16.         mutexCounter.unlock();
17.     }
18. }
19.
20. int main(){
21.     std::thread thread1(increment);
22.     std::thread thread2(increment);
23.
24.     thread1.join();
25.     thread2.join();
26.
27.     std::cout << "Counter: " << counter << std::endl;
28.
29.     return 0;
30. }
```

⑤

g++ -std=c++11 program.cpp -o program

flag
fuente
nombre del binario

PThreads syntax

```

1.  #include <iostream>
2.  #include <pthread.h>
3.
4.  void* task(void* arg){
5.      std::cout << "Hello from the thread!" << std::endl;
6.      return NULL;
7.  }
8.
9.  int main(){
10.     pthread_t thread;
11.     pthread_create(
12.         &thread,
13.         NULL,
14.         task,
15.         NULL
16.     );
17.
18.     pthread_join(thread, NULL);
19.     return 0;
20. }

```

①

```

1.  #include <iostream>
2.  #include <pthread.h>
3.
4.  void* printNumber(void* arg){
5.      int* number = (int*)arg;
6.      std::cout << "Number: " << *number << std::endl;
7.      return NULL;
8.  }
9.
10. int main()
11. {
12.     pthread_t thread;
13.     int number = 100;
14.
15.     pthread_create(
16.         &thread,
17.         NULL,
18.         printNumber,
19.         &number
20.     );
21.     pthread_join(thread, NULL);
22.
23.     return 0;
24. }

```

②

g++ program.cpp -o program -pthread

Flag



```

1.  #include <iostream>
2.  #include <pthread.h>
3.
4.  void* task(void* arg)
5.  {
6.      int* id = (int*)arg;
7.
8.      std::cout << "Thread " << *id << std::endl;
9.
10.     return NULL;
11. }
12.
13. int main()
14. {
15.     const int numThreads = 4;
16.
17.     pthread_t threads[numThreads];
18.     int ids[numThreads];
19.
20.     for (int i = 0; i < numThreads; ++i) {
21.
22.         ids[i] = i;
23.
24.         pthread_create(
25.             &threads[i],
26.             NULL,
27.             task,
28.             &ids[i]
29.         );
30.     }
31.
32.     for (int i = 0; i < numThreads; ++i) {
33.         pthread_join(threads[i], NULL);
34.     }
35.
36.     return 0;
37. }

```

③

OpenMP syntax

```

1. #include <iostream>
2. #include <omp.h>
3.
4. int main() {
5.     #pragma omp parallel {
6.         std::cout << "Hello from a thread\n";
7.     }
8.     return 0;
9. }
```

①

g++ -fopenmp program.cpp -o program

```

1. #include <iostream>
2. #include <omp.h>
3.
4. int main() {
5.     const int n = 16;
6.     int numbers[n];
7.
8.     for (int i = 0; i < n; ++i)
9.         numbers[i] = i + 1;
10.
11.     #pragma omp parallel for
12.     for (int i = 0; i < n; ++i)
13.     {
14.         int id = omp_get_thread_num();
15.
16.         std::cout << "Thread "
17.                   << id
18.                   << " processes index "
19.                   << i << std::endl;
20.     }
21.
22.     return 0;
23. }
```

②

```

1. #include <iostream>
2. #include <omp.h>
3.
4. int main()
5. {
6.     const int n = 16;
7.
8.     int numbers[n] = {
9.         1, 2, 3, 4,
10.        5, 6, 7, 8,
11.        9, 10, 11, 12,
12.        13, 14, 15, 16
13.    };
14.
15.    long long sum = 0;
16.
17.    omp_set_num_threads(4);
18.
19.    #pragma omp parallel for reduction(+:sum)
20.    for (int i = 0; i < n; ++i)
21.    {
22.        sum += numbers[i];
23.    }
24.
25.    std::cout << "Sum: " << sum << std::endl;
26.
27.    return 0;
28. }
```

③

```

int x = 10;

#pragma omp parallel private(x) {
    x = omp_get_thread_num();
    std::cout << x << std::endl;
}

...
#pragma omp parallel shared(numbers)

...
#pragma omp critical

...
```

Exercise

Implement program that calculates the sum of all elements in an integer array using Threads (*Pthreads*, *Threads* and *OpenMP*).

Strategy 1 — Block Partitioning

Divide the array into contiguous blocks, assigning one block to each thread.

```
Thread 0 → elements 0 - 3
Thread 1 → elements 4 - 7
Thread 2 → elements 8 - 11
Thread 3 → elements 12 - 15
```

Strategy 2 — Cyclic / Interleaved Partitioning

Instead of assigning contiguous blocks, distribute the elements cyclically among the threads.

```
Thread 0 → elements 0, 4, 8, 12
Thread 1 → elements 1, 5, 9, 13
Thread 2 → elements 2, 6, 10, 14
Thread 3 → elements 3, 7, 11, 15
```

1. Use japeto/parallel-tools Docker image
2. Use the following function names:
 - a. `sumBlock` for Block Partitioning strategy
 - b. `sumCyclic` for Cyclic Partitioning strategy
3. Use the following variables:
 - a. `numbers` // input array
 - b. `numThreads` // number of threads
 - c. `threads` // pthread_t array
 - d. `partialSums` // partial results

Goal: demonstrate that the same problem can be parallelized using two different data-distribution strategies